Probabilistic Pairing Proofs

Shramee Srivastav
[0009-0009-7208-3478]
MIST.cash—FOCBB, Shhtarknet
shramee@proton.me

April 8, 2026

Abstract

Elliptic curve pairings are fundamental to modern zero-knowledge proof systems, including Groth16, PLONK, and KZG polynomial commitments. In resource-constrained execution environments (Ethereum VM, BitVM, blockchain smart contracts) where computation is metered by gas fees, and in arithmetized computation systems (R1CS, Plonkish, Cairo AIR) where each computational step imposes significant prover overhead, the cost of pairing computation remains a practical bottleneck.

We present a public-coin interactive proof system that consolidates the Miller loop iterations and final exponentiation into a single polynomial relation. The relation can then be verified as polynomial identities greatly reducing the pairing costs.

The techniques presented here have been integrated into Garaga [FE], a production elliptic curve tooling library for Cairo and Starknet. For BN254 3-pairing verification (standard in Groth16), the implementation achieves: 45% reduction in field operations (14,456 → 7,975 MULMOD), 77% reduction in commitment overhead (3,758 → 876 POSEIDON hashes), and 37% overall speedup. These improvements enable practical pairing verification in resource-constrained blockchain environments and recursive proof systems.

Contents

List of Algorithms

List of Tables

1 Introduction

Blockchain technology continues to mature with improving scalability and infrastructure. However, privacy remains a significant barrier to entry for many users and applications. Zero-knowledge proofs (ZKPs) have emerged as a powerful solution to this challenge.

1.1 Motivation

Elliptic curve pairings represent fundamental building blocks for numerous cryptographic systems including Groth16 [Gro16], PLONK [GWC19], BLS signature schemes [BGW+22], and KZG polynomial commitment schemes [KZG10].

However, **pairings are computationally intensive**. This creates critical bottlenecks in two important deployment contexts:

1. **Resource-constrained virtual machines** (Ethereum VM, BitVM, ZKVMs): Computation is metered — each operation incurs gas costs. A single pairing verification requiring thousands of field operations becomes economically prohibitive during peak network congestion.

2. **Arithmetized computation systems** (R1CS [GGPR13], Plonkish [GWC19], Cairo AIR [GPR21]): Each operation must be represented algebraically, adding significant prover overhead beyond native execution costs.

In such contexts, an efficient proof system for pairing correctness can replace expensive native execution inside the constrained environment.

1.2 Our Contribution

We present a public-coin interactive proof system for verifying pairing computations that consolidates verification into unified polynomial relationships under the Schwartz-Zippel lemma. By treating complete Miller loop iterations as single polynomial relationships rather than sequences of individual extension field operations [Fel23], our approach achieves:

- **45% reduction** in Miller loop field operations

- **77% reduction** in commitment overhead (Fiat-Shamir hashing)
- **37% overall speedup** for complete pairing verification

Our key insight is that instead of verifying individual field operations separately, we can verify entire Miller loop iterations as single polynomial identities. This consolidation dramatically reduces both the number of modular operations required and the communication overhead from intermediate commitments.

2 Background

Algorithm 1 Optimal Ate pairing over Barreto-Naehrig curves [BGM+10]

Input: $P \in \mathbb{G}_1, Q \in \mathbb{G}_2$
Output: $a_{opt}(Q, P)$

Write $s = 6t + 2 = \sum_{i=0}^{L-1} s_i 2^i$ where $s_i \in \{-1, 0, 1\}$ and $s_L = 1$
$T \leftarrow Q, f \leftarrow 1$
for $i = L - 2$ to 0 **do**
    $f \leftarrow f^2 \cdot l_{T,T}(P)$
    $T \leftarrow [2]T$
    if $s_i = 1$ **then**
        $f \leftarrow f \cdot l_{T,Q}(P)$
        $T \leftarrow T + Q$
    end if
    if $s_i = -1$ **then**
        $f \leftarrow f \cdot l_{T,-Q}(P)$
        $T \leftarrow T - Q$
    end if
end for
$Q_1 \leftarrow \pi_p(Q), Q_2 \leftarrow \pi_p^2(Q)$
$f \leftarrow f \cdot l_{T,Q_1}(P), T \leftarrow T + Q_1$
$f \leftarrow f \cdot l_{T,-Q_2}(P), T \leftarrow T - Q_2$
$f \leftarrow f^{(p^{12}-1)/r}$
return f

2.1 Pairing Computation

An elliptic curve pairing is a non-degenerate bilinear map $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$, where $\mathbb{G}_1$ and $\mathbb{G}_2$ are groups of points on elliptic curves over finite fields, and $\mathbb{G}_T$ is a multiplicative group in an extension field. For practical cryptographic applications, we require that this map be efficiently computable. We will focus on the optimal ate pairing [BGM+10] over Barreto-Naehrig (BN) curves, which is widely used due to its efficiency. We will specifically use the Ethereum's BN254 curve, defined over a 254-bit prime field $\mathbb{F}_q$, with groups $\mathbb{G}_1, \mathbb{G}_2$ of order r and target group $\mathbb{G}_T$ contained in the 12th extension field $\mathbb{F}_{q^{12}}$.

The computation of an optimal ate pairing—the focus of our implementation—consists of two primary phases:

1. **Miller loop**: The core iterative computation that evaluates line functions at pairs of points from $\mathbb{G}_1$ and $\mathbb{G}_2$, accumulating intermediate results to produce a value in $\mathbb{F}_{q^{12}}$.

2. **Final Exponentiation**: This step raises the Miller loop output to the power $(q^{12}-1)/r$, where r is the order of the groups, ensuring the result lies in the correct subgroup $\mathbb{G}_T$.

While both phases present optimization opportunities, the constraint-based verification of these computations introduces unique challenges and possibilities that differ significantly from native implementations.

2.2 Schwartz-Zippel lemma

The Schwartz-Zippel lemma (or more precisely, the Demillo-Lipton-Schwartz-Zippel lemma [DL78, Sch79, Zip79, Sch80]) provides a probabilistic method for testing polynomial identities. The lemma states that for a non-zero polynomial $P(x_1, x_2, \ldots, x_n)$ of total degree d over a finite field $\mathbb{F}$, when evaluated at uniformly random field elements, the probability that P evaluates to zero is at most $d/|\mathbb{F}|$.

This probabilistic bound becomes negligible for cryptographically sized fields where $d \ll |\mathbb{F}|$, making it practical to verify polynomial identities by evaluation instead of coefficient-wise operations.

2.3 Computation Models

While we specifically implement our library in Cairo, we map our MULMOD operation to a constraint (denoted by **C**) to maintain compatibility with the Rank-1 Constraint System (R1CS) model to benchmark against established literature. In R1CS [Gro16], prover complexity is dominated by multiplication gates while additions are free. In the Cairo VM, this corresponds to the Algebraic Intermediate Representation (AIR) where operations are verified through polynomial constraints. To synchronize our data with the benchmarks provided by El Housni [Hou23], we define our cost metric based on field multiplications.

2.3.1 R1CS Cost Model

We measure computational cost in terms of *constraints* (denoted by **C**), representing the fundamental unit of work in Rank-1 constraint systems. One constraint corresponds to one multiplication gate.

Basic Field Operations in $\mathbb{F}_q$:

- **Addition/Subtraction:** Free
- **Multiplication:** One constraint (1C)
- **Division/Inversion:** One multiplication + one equality check

Commitment Overhead: Beyond constraint costs, we account for *calldata* or *commitment overhead*—the number of field elements the prover must commit to (via Fiat-Shamir hashing in non-interactive proofs). For blockchain deployments, this overhead directly impacts transaction costs and is often the dominant factor.

2.3.2 Cost Model Comparison

Table 1 contrasts costs between native implementations and constraint systems:

Table 1: Cost comparison: Native vs Constraint-based computation

Operation	Native Cost	Constraint Cost	Ratio
$\mathbb{F}_q$ Multiplication	1×	1C	1×
$\mathbb{F}_q$ Inversion	100×	2C	≈50× cheaper
$\mathbb{F}_{q^{12}}$ Multiplication	50×	54C (traditional)	Similar
$\mathbb{F}_{q^{12}}$ Multiplication	50×	39C (our method)	Better

Key Insight: Operations that are traditionally expensive (inversions, divisions) become cheap in constraint systems when computed via witnesses. This inverts traditional optimization priorities: we can afford inversions to save multiplications, enabling affine coordinate systems and other techniques impractical in native implementations.

Recognizing and exploiting these unique computational properties is essential for unlocking hidden performance potential. Youssef El Housni's pioneering work **Pairings in Rank-1 Constraint Systems** [Hou23] represents, to our knowledge, the first systematic exploration of these optimization opportunities, providing a breakthrough in the practicality of recursive proving by thoughtfully leveraging the distinctive cost model inherent to constraint-based systems.

3 Related Work

The landscape of efficient pairing verification in constraint systems has evolved through three key contributions that our work directly builds upon.

El Housni's **Pairings in Rank-1 Constraint Systems** [Hou23] established the theoretical framework for optimizing pairings in arithmetized environments, introducing efficient Miller loop formulas in affine coordinates and sparse line function representations. This work inverts the usual cost model: inversions become cheap (2C) while multiplications remain the dominant cost.

Feltroidprime [Fel23] introduced polynomial identity testing for $\mathbb{F}_{q^{12}}$ arithmetic via the Schwartz-Zippel lemma. Each multiplication $A \cdot B$ is expressed as $A(x) \cdot B(x) = Q(x) \cdot P_{12}(x) + R(x)$ and verified at a single random point, with the quotients Q batched via random linear combination. Our work applies the same polynomial framework but at a coarser granularity: entire Miller loop steps rather than individual multiplications, eliminating intermediate remainder commitments entirely.

Novakovic and Eagen's **On Proving Pairings** [NE24] showed that final exponentiation can be replaced by a residue witness check, reducing final exponentiation cost by over 85% for BN254. We integrate their residue witness c directly into our polynomial step equations (Section 4.2.2), unifying the Miller loop and final exponentiation into the same polynomial verification framework.

4 Pairing Operations as Polynomials

Algorithm 2 gives the pairing computation combining El Housni's R1CS optimizations [Hou23] with Novakovic-Eagen's residue witness technique [NE24]. The 27th root

of unity W scales the Miller loop output f to be a cubic residue (see [NE24] §4.3); if f is not already a cubic residue, multiplying by W^w, $w \in \{1, 2\}$ corrects this.

Our contribution is to express each step of this algorithm as a single multi-operand polynomial product verified via the Schwartz-Zippel lemma, rather than as a sequence of individual binary multiplications as in [Fel23].

Algorithm 2 Optimal Ate pairing with Residue witness check

Input: $P \in \mathbb{G}_1, Q \in \mathbb{G}_2, c \in \mathbb{F}_{q^{12}}, w \in \{0, 1, 2\}$
Internal $W, W^2 \in \mathbb{F}_{q^{12}}$ ▷ W is 27th root of unity
Output: $a_{opt}(Q, P)$

1: Write $s = 6t + 2 = \sum_{i=0}^{L-1} s_i 2^i$ where $s_i \in \{-1, 0, 1\}$ and $s_L = 1$
2: $c^{-1} \leftarrow 1/c$ ▷ Inverse Residue witness
3: $f \leftarrow c^{-1}$ ▷ Initialize with inverse residue witness
4: $T \leftarrow Q$ ▷ T_i for each pair for multi-pairing
Every operation involving T, P or Q is repeated for T_i, P_i and Q_i for multi-pairing.

5: **for** $i = L - 2$ to 0 **do**
6: $f \leftarrow f^2$
7: $f \leftarrow f \cdot l_{T,T}(P)$ ▷ Repeat for T_i, P_i
8: $T \leftarrow [2]T$ ▷ Repeat for T_i
9: **if** $s_i = 1$ **then**
10: $f \leftarrow f \cdot c^{-1}$ ▷ Residue witness
11: $f \leftarrow f \cdot l_{T,Q}(P)$ ▷ Repeat for T_i, Q_i, P_i
12: $T \leftarrow T + Q$ ▷ Repeat for T_i, Q_i
13: **else if** $s_i = -1$ **then**
14: $f \leftarrow f \cdot c$ ▷ Residue witness
15: $f \leftarrow f \cdot l_{T,-Q}(P)$ ▷ Repeat for $T_i, -Q_i, P_i$
16: $T \leftarrow T - Q$ ▷ Repeat for $T_i, -Q_i$
17: **end if**
18: **end for**

19: $Q_1 \leftarrow \pi_p(Q), Q_2 \leftarrow \pi_p^2(Q)$ ▷ Repeat for Q_{1i}, T_i
20: $f \leftarrow f \cdot l_{T,Q_1}(P)$ ▷ Repeat for Q_{1i}, T_i, P_i
21: $T \leftarrow T + Q_1$ ▷ Repeat for Q_{1i}, T_i
22: $f \leftarrow f \cdot l_{T,-Q_2(Q)}(P)$ ▷ Repeat for Q_i, T_i, P_i
▷ Skip $T \leftarrow T - Q_2(Q)$

23: **if** $w = 1$ **then**
24: $f \leftarrow f \cdot W$
25: **else if** $w = 2$ **then**
26: $f \leftarrow f \cdot W^2$
27: **end if**
28: $f \leftarrow f \cdot c^{-q} \cdot c^{q^2} \cdot c^{-q^3}$ ▷ Uses Frobenius maps, Negative exponents use c^{-1}

29: **return** f

4.1 Comparative Analysis

We compare the two schemes for a 3-pair Miller loop zero-bit step (the most common case in BN254's NAF [DHM04] representation of $s = 6t + 2$). Here $F \in \mathbb{F}_{q^{12}}$ is the

accumulator, $L_i \in \text{Sparse}_{01379}$ are the three doubling line functions, $R \in \mathbb{F}_{q^{12}}$ is the updated accumulator, and Q is the quotient polynomial.

4.1.1 Previous scheme

Adapting faster extension field multiplication [Fel23] idea to line 6 and 7 of Miller loop algorithm 2 for a 3–pair Miller loop, we have following polynomial relationships to assert.

$$
\begin{aligned}
F(x) \cdot F(x) &\stackrel{?}{=} Q_1(x) \cdot P_{12}(x) + T_1(x) \\
T_1(x) \cdot L_1(x) &\stackrel{?}{=} Q_2(x) \cdot P_{12}(x) + T_2(x) \\
T_2(x) \cdot L_2(x) &\stackrel{?}{=} Q_3(x) \cdot P_{12}(x) + T_3(x) \\
T_3(x) \cdot L_3(x) &\stackrel{?}{=} Q(x) \cdot P_{12}(x) + R(x)
\end{aligned}
$$

F, T_1, T_2, T_3, R cost 12C each for evaluation, 4C for $\mathbb{F}_q$ multiplication, and L_1, L_2, L_3 require 4C each for evaluation. All the remainder hints need to be provided for commitment, so this is 12 calldata commitment for R and intermediate remainder polynomials T_1, T_2, T_3. **Costs sum up to 76C** ($5 * 12C + 3 * 4C + 4C$) **and 48 commitments** in $\mathbb{F}_q$ $(4 * 12)$. Resulting remainder polynomial $R \in \mathbb{F}_{q^{12}}$ is the result of the Miller loop step.

4.1.2 Our scheme

In our approach, instead of working with individual arithmetic operations, we treat the entire bit operation as a single polynomial. In our scheme line 6 and 7 of 3–pair Miller loop (Algorithm 2) looks like this.

$$
F(x) \cdot F(x) \cdot L_1(x) \cdot L_2(x) \cdot L_3(x) \stackrel{?}{=} Q(x) \cdot P_{12}(x) + R(x)
$$

Similar to the previous approach, F and R cost 12C each for evaluation, 4C for $\mathbb{F}_q$ multiplication, and L_1, L_2, L_3 require 4C each for evaluation, but we eliminate intermediate (T_* polynomials above) commitments and evaluations. Only the remainder hint needs commitment, so this is 12 calldata commitment for R. **Costs sum up to 41C** ($2 \times 12\text{C} + 3 \times 4\text{C} + 4\text{C} + 1\text{C}$ (RLC)) **and 12 $\mathbb{F}_q$ commitments**.

This results in **46% fewer constraints** and a **75% reduction in commitments**. When verifying ZK proofs on-chain, commitment savings are often more critical than constraint savings, due to the calldata costs associated with transmitting polynomial hints.

4.1.3 Performance Comparison

Table 2 compares the computational costs and commitment overhead between the schemes for a single 0-bit Miller loop operation with 3 pairs.

Table 2: 3-Pair Miller loop 0-bit constraints comparison

Scheme	Constraints	$\mathbb{F}_q$ Commitments	Improvement
[Hou23]	132C	—	Baseline
[Fel23]	76C*	48	42%
Our scheme	**41C***	**12**	**69%**

[Hou23]: 1 square (36C) + 3 sparse mul (3×30C) = 132C
[Fel23]: 5 evals (5×12C) + 3 line evals (3×4C) + 4 muls (4×1C) = 76C
Our scheme: 2 evals (2×12C) + 3 line evals (3×4C) + 5C (muls + RLC) = 41C
*Costs exclude Fiat-Shamir commitment overhead

4.2 Operation Polynomials

Our implementation uses different polynomials for different bits in the Miller loop. Let us walk through a 3-pairing setup which Inversions used widely for Groth16 verification. We will describe the polynomials for zero bits, non-zero bits and correction step.

4.2.1 Miller loop: Zero Bit Equation

As seen in 4.1.2 for zero bit operations (line 6 and 7 with three $l_{T,T}(P)$ lines in Algorithm 2), the verification equation is:

$$F^2(x) \cdot L_1(x) \cdot L_2(x) \cdot L_3(x) = R(x) + Q(x) \cdot P_{12}(x) \tag{1}$$

Polynomial	Description	Degree	Coefficients
$F \in \mathbb{F}_{q^{12}}$	Miller loop accumulator	$11 \times 2 = 22$	12
$L_1, L_2, L_3 \in \text{Sparse}_{01379}$	The line functions	$9 \times 3 = 27$	$4 \times 3 = 12$
$R \in \mathbb{F}_{q^{12}}$	Remainder, The result	11	12
Total constraints	36C Evals, 4C Muls and 1C RLC		**41**
Q *(Amortized See 5.1)*	$\deg(\text{LHS}) - \deg(P_{12}) + 1$	38	39

Table 3: Zero bit operation polynomial components

4.2.2 Miller loop: Non-Zero Bit Equation

For non-zero bit operations(lines 10, 11, 14, 15 with three pairs of lines in Algorithm 2), the Miller loop performs both doubling and addition steps. So for each pair we have two Sparse_{01379} lines. The verification equation becomes:

$$F^2(x) \cdot W(x) \cdot L_{\text{d1}}(x) \cdot L_{\text{d2}}(x) \cdot L_{\text{d3}}(x) \cdot L_{\text{a1}}(x) \cdot L_{\text{a2}}(x) \cdot L_{\text{a3}}(x) = R(x) + Q(x) \cdot P_{12}(x) \tag{2}$$

The residue witness W encapsulates the equivalence class relationship established by Novakovic and Eagen's final exponentiation elimination. The inverse of W is used for negative bits, this just changes the coefficients the equation remains the same.

Polynomial	Description	Degree	Coefficients
$F \in \mathbb{F}_{q^{12}}$	Miller loop accumulator	$11 \times 2 = 22$	12
$L_{\text{d1}}, L_{\text{d2}}, L_{\text{d3}} \in \text{Sparse}_{01379}$	Doubling line functions	$9 \times 3 = 27$	$4 \times 3 = 12$
$L_{\text{a1}}, L_{\text{a2}}, L_{\text{a3}} \in \text{Sparse}_{01379}$	Addition line functions	$9 \times 3 = 27$	$4 \times 3 = 12$
W or $W^{-1} \in \mathbb{F}_{q^{12}}$	Residue witness	11	12
$R \in \mathbb{F}_{q^{12}}$	Remainder, The result	11	12
Total constraints	60C Evals, 8C Muls and 1C RLC		**69**
Q *(Amortized See 5.1)*	$\deg(\text{LHS}) - \deg(P_{12}) + 1$	76	77

Table 4: Non-zero bit operation polynomial components

4.2.3 Miller loop: Frobenius mapping Equation

The correction step finalizes the Miller loop by incorporating the Frobenius endomorphism corrections (lines 20 and 22 in Algorithm 2). Again, we have two Sparse_{01379} lines for each pair, for π_p and $-\pi_p^2$ mappings, without any squaring of the accumulator. The verification equation becomes:

$$F(x) \cdot L_{1\pi}(x) \cdot L_{2\pi}(x) \cdot L_{3\pi}(x) \cdot L_{1\pi^2}(x) \cdot L_{2\pi^2}(x) \cdot L_{3\pi^2}(x) = R(x) + Q(x) \cdot P_{12}(x) \quad (3)$$

Polynomial	Description	Degree	Coefficients
$F \in \mathbb{F}_{q^{12}}$	Miller loop accumulator	11	12
$L_{1\pi}, L_{2\pi}, L_{3\pi} \in \text{Sparse}_{01379}$	π_p correction lines	$9 \times 3 = 27$	$4 \times 3 = 12$
$L_{1\pi^2}, L_{2\pi^2}, L_{3\pi^2} \in \text{Sparse}_{01379}$	$-\pi_p^2$ correction lines	$9 \times 3 = 27$	$4 \times 3 = 12$
$R \in \mathbb{F}_{q^{12}}$	Remainder, The result	11	12
Total constraints	48C Evals, 6C Muls and 1C RLC		**55**
Q *(Amortized See 5.1)*	$\deg(\text{LHS}) - \deg(P_{12}) + 1$	54	55

Table 5: Correction step polynomial components

4.2.4 Final Exponentiation Equation

The final exponentiation check is handled as described in [NE24]. The equation handles the cubic scale factor multiplication, and **Frobenius component** $(q - q^2 + q^3)$ of the final exponentation (lines 24, 26 and 28 in Algorithm 2). The verification equation looks like:

$$F(x) \cdot c^{-q}(x) \cdot c^{q^2}(x) \cdot c^{-q^3}(x) \cdot W^w(x) = 1 + Q(x) \cdot P_{12}(x) \quad (4)$$

where $w \in \{0, 1, 2\}$ determines the cubic root of unity scaling factor, and the Frobenius mappings are computed using the residue witness c and its inverse and provided as input. Cubic root is sparse element with just 2 coefficients the degree ranging from 8 to 10 because of positioning of the coefficients. Our R is now 1 for a successful proof verification.

With this we have all the polynomials we need for verifying entire pairing operation. Next we will look into high level overview of pairing verification via an interactive oracle proof.

Polynomial	Description	Degree	Coefficients
$F \in \mathbb{F}_{q^{12}}$	Final Miller loop result	11	12
$c^{-q} \in \mathbb{F}_{q^{12}}$	Residue witness Frobenius $-q$	11	12
$c^{q^2} \in \mathbb{F}_{q^{12}}$	Residue witness Frobenius q^2	11	12
$c^{-q^3} \in \mathbb{F}_{q^{12}}$	Residue witness Frobenius $-q^3$	11	12
$W^w \in \mathbb{F}_{q^{12}}$	Cubic scale factor	10	2
$R = 1$	Unity (constant)	0	1
Total constraints	51C Evals, 4C Muls and 1C RLC		**56**
Q *(Amortized See 5.1)*	$\deg(\text{LHS}) - \deg(P_{12}) + 1$	43	44

Table 6: Final exponentiation polynomial components

4.3 Security

Each verification equation takes the form $P = \prod_i A_i(x) - Q(x) \cdot P_{12}(x) - R(x)$ where $\deg(P) < 2^8$.[1] By the Schwartz-Zippel lemma (Section 2.2):

Soundness: $\delta_s \leq d/|\mathbb{F}_q| < 2^8/2^{254} = 2^{-246}$ for BN254.

Completeness: An honest prover always produces $P(x) = 0$ identically, so the verifier always accepts. Euclidean division guarantees unique Q, R with $\deg(R) < \deg(P_{12}) = 12$.

5 Full Pairing Proof with Quotient Batching

We now combine the polynomial equations from Section 4.2 into a complete two-round public-coin interactive proof, further reducing verification cost by batching all quotients into a single accumulated polynomial.

5.1 Batching Polynomial Identity Testing

All n polynomial equations from Section 4.2 are batched into a single verification. The prover commits to remainders $\{R_i\}_{i=0}^{n-1}$ and receives challenge z. Using z, the prover forms the accumulated quotient $Q_{\text{acc}} = \sum_{i=0}^{n-1} z^i \cdot Q_i$. The verifier then samples a second challenge point c and checks:

$$\sum_{i=0}^{n-1} z^i \cdot [A_i(c) \cdot B_i(c) \cdots X_i(c) - R_i(c)] = Q_{\text{acc}}(c) \cdot P_{12}(c) \tag{5}$$

This reduces n polynomial verifications (degrees 38–76, see Tables 3–6) to a single evaluation of a degree-76 polynomial.

5.2 Prover's Algorithms

The prover computes quotients Q_i and remainders R_i for each equation in Section 4.2. For each step, it calls PolyMulDiv (Algorithm 3), which multiplies the input polynomials and performs standard Euclidean division by the irreducible P_{12}. After receiving challenge

[1] For k-pair pairings, $\deg(P) \approx 88 + 18(k-3)$, well below 2^8 for all practical k. This bound can be extended to 2^{10} with negligible soundness impact.

z, the prover forms $Q_{\text{acc}} = \sum_i z^i Q_i$ and sends $\mathcal{R} = \{R_i\}$ and Q_{acc} to the verifier. The full prover procedure is given in Algorithm 4.

Algorithm 3 PolyMulDiv: Multiply polynomials and divide by P_{12}

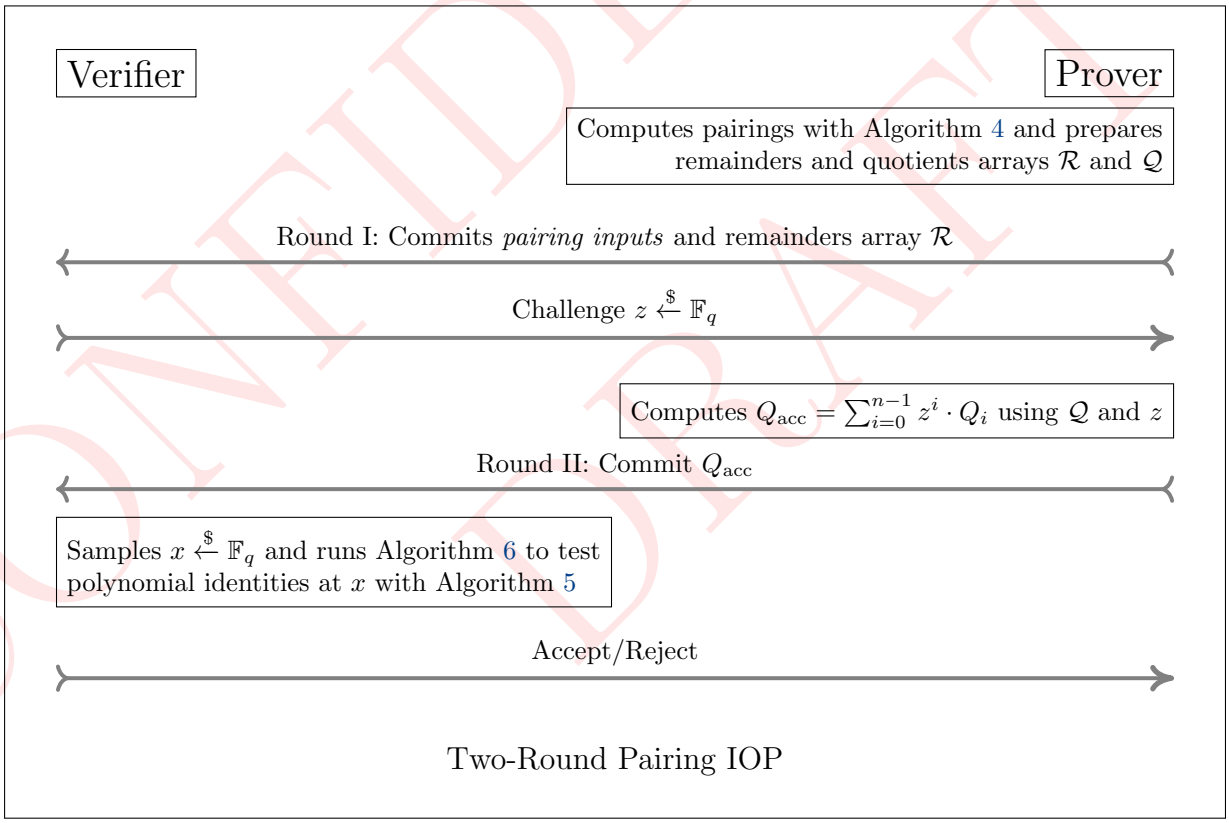
Input: $P_1, \ldots, P_n \in \mathbb{F}_q[x]$; Divisor $P_{12} \in \mathbb{F}_q[x]$
Output: Q, R s.t. $\prod P_i = Q \cdot P_{12} + R$, $\deg(R) < \deg(P_{12})$

1: $R \leftarrow \prod_{i=1}^{n} P_i$ ▷ Polynomial product
2: Perform Euclidean division of R by P_{12} to get (Q, R)
3: **return** (Q, R)

5.3 Verifier's Algorithms

The verifier receives $P \in \mathbb{G}_1$, $Q \in \mathbb{G}_2$, residue witness c [NE24], the W (27th root of unity), the remainder array $\mathcal{R}$, and Q_{acc} from the prover. Using challenge z from the first round and a freshly sampled x, it mirrors the prover's pairing loop while evaluating each polynomial at x via Algorithm 5, accumulating the RLC e_{acc}. The final check is Equation 5. The full verifier is given in Algorithm 6.

5.4 Interactive Proof



5.5 Fiat-Shamir Transformation

Applying the Fiat-Shamir transformation [FS87] with random oracle H gives a non-interactive proof with knowledge soundness $\delta_s + 3(m^2+1)2^{-\lambda}$ [BSCS16], where $m = 2$

Algorithm 4 Pairing Prover: Preparing polynomials

Input: $P \in \mathbb{G}_1, Q \in \mathbb{G}_2, c \in \mathbb{F}_{q^{12}}, w \in \{0, 1, 2\}$
Internal: $W, W^2 \in \mathbb{F}_{q^{12}}$ ▷ W is 27th root of unity
Output: Arrays $\mathcal{Q}, \mathcal{R}$ of quotients and remainders
Write $s = 6t + 2 = \sum_{i=0}^{L-1} s_i 2^i$ where $s_i \in \{-1, 0, 1\}$ and $s_L = 1$

$c^{-1} \leftarrow 1/c$ ▷ Inverse Residue witness
$f \leftarrow c^{-1}$ ▷ Initialize with inverse residue witness
$T \leftarrow Q$ ▷ T_i for each pair for multi-pairing
Every operation involving T, P or Q is repeated for T_i, P_i and Q_i for multi-pairing.

Miller loop bits
for $i = L - 2$ to 0 **do**
  $L_{\text{dbl}} \leftarrow l_{T,T}(P)$, $T \leftarrow [2]T$
  if $s_i = 0$ **then**
    $Q, R \leftarrow PolyMulDiv(f, f, L_{\text{dbl}})$ ▷ Algorithm 3, add L_{*i} for each P_i, Q_i
  else if $s_i = 1$ **then**
    $L_{\text{add}} \leftarrow l_{T,Q}(P)$, $T \leftarrow T + Q$
    $Q, R \leftarrow PolyMulDiv(f, f, c^{-1}, L_{\text{dbl}}, L_{\text{add}})$ ▷ Algorithm 3, add L_{*i} for each P_i, Q_i
  else if $s_i = -1$ **then**
    $L_{\text{add}} \leftarrow l_{T,-Q}(P)$, $T \leftarrow T - Q$
    $Q, R \leftarrow PolyMulDiv(f, f, c, L_{\text{dbl}}, L_{\text{add}})$ ▷ Algorithm 3, add L_{*i} for each P_i, Q_i
  end if
  $f \leftarrow R$, $\mathcal{Q}.append(Q)$, $\mathcal{R}.append(R)$
end for

Miller loop Frobenius correction
$Q_1 \leftarrow \pi_p(Q), Q_2 \leftarrow \pi_p^2(Q)$
$L_\pi \leftarrow l_{T,Q_1}(P)$
$T \leftarrow T + Q_1$
$L_{-\pi^2} \leftarrow l_{T,-Q_2}(P)$
$Q, R \leftarrow PolyMulDiv(f, L_\pi, L_{-\pi^2})$ ▷ Algorithm 3, add L_{*i} for each P_i, Q_i
$f \leftarrow R$, $\mathcal{Q}.append(Q)$, $\mathcal{R}.append(R)$

Final exponentiation: Cubic scaling + Frobenius mappings
if $w = 0$ **then**
  $Q, R \leftarrow PolyMulDiv(f, c^{-q}, c^{q^2}, c^{-q^3})$ ▷ Algorithm 3
else if $w = 1$ **then**
  $Q, R \leftarrow PolyMulDiv(f, W, c^{-q}, c^{q^2}, c^{-q^3})$ ▷ Algorithm 3
else if $w = 2$ **then**
  $Q, R \leftarrow PolyMulDiv(f, W^2, c^{-q}, c^{q^2}, c^{-q^3})$ ▷ Algorithm 3
end if
$\mathcal{Q}.append(Q)$, $\mathcal{R}.append(R)$ ▷ Final $\mathcal{Q}$ and $\mathcal{R}$ accumulation, skip f

return $\mathcal{Q}, \mathcal{R}$

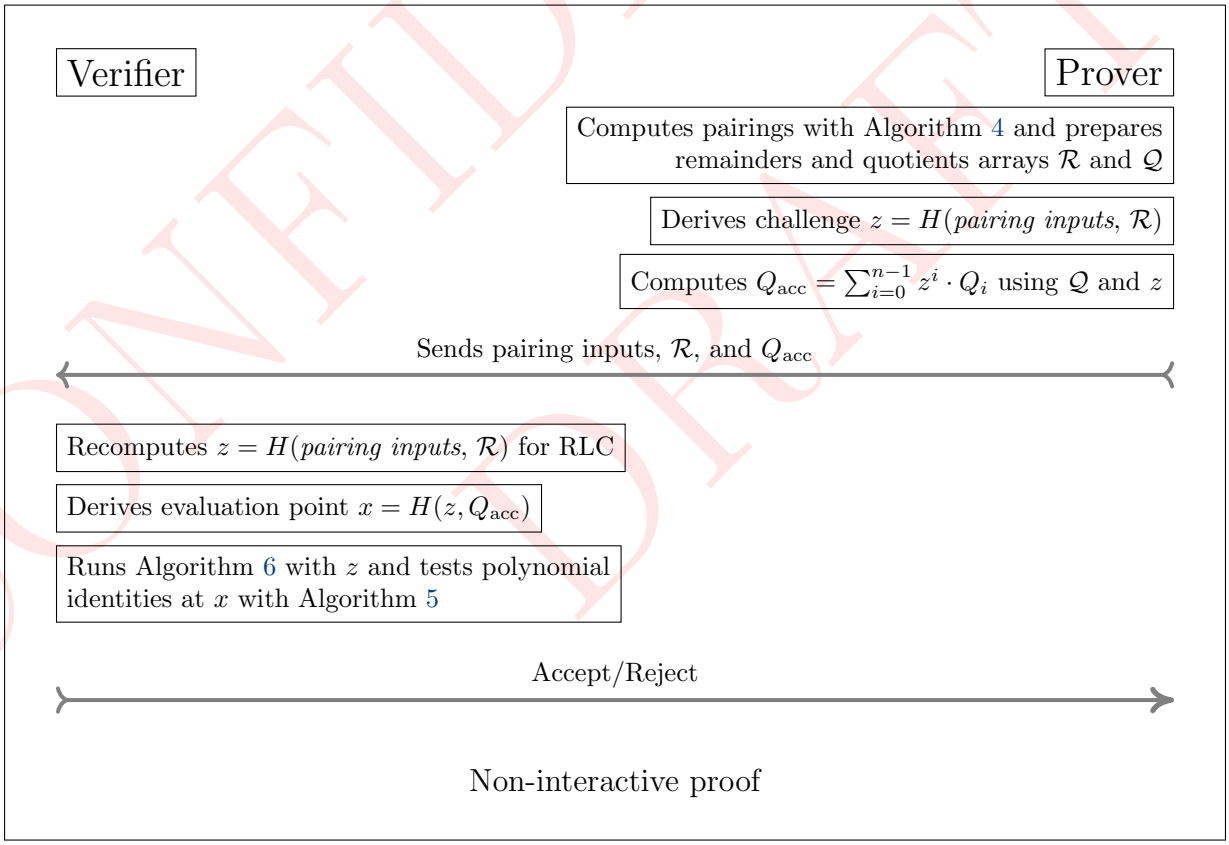
Algorithm 5 EvalPoly: Evaluate polynomial product minus remainder via Horner's method

Input: Polynomials $P_1, \ldots, P_n$, $R \in \mathbb{F}_q[x]$ of degree $\leq d$, evaluation point $z \in \mathbb{F}_q$
Output: $e \in \mathbb{F}_q$ where $e = \prod_{i=1}^{n} P_i(z) - R(z)$

function EVAL($P = (p_0, p_1, \ldots, p_d)$, z) ▷ Uses Horner's method
  $v \leftarrow p_d$ ▷ Start from leading coefficient
  for $k = d - 1$ **downto** 0 **do**
    $v \leftarrow v \cdot z + p_k$ ▷ d multiplications, d additions total
  end for
  return v
end function
$e \leftarrow$ EVAL(P_1, z) ▷ Evaluate first polynomial
for $i = 2$ to n **do**
  $e \leftarrow e \cdot$ EVAL(P_i, z) ▷ Multiply in each subsequent evaluation
end for
$e \leftarrow e -$ EVAL(R, z) ▷ Subtract remainder evaluation
return e

(hash queries) and $\lambda = 254$ (BN254 hash output length).

5.6 Non-Interactive Proof



Concrete Fiat-Shamir costs: In our Cairo implementation, each POSEIDON hash invocation costs approximately 25-30 Cairo steps. Our batching strategy reduces the number of hash operations from $O(k \cdot n)$ to $O(k)$ where k is the Miller loop length and

Algorithm 6 Pairing Verifier: Evaluating remainders

Input: $P \in \mathbb{G}_1, Q \in \mathbb{G}_2, c \in \mathbb{F}_{q^{12}}, w \in \{0,1,2\}$, $\mathcal{R}$ array of remainders, $Q_{\text{acc}} \in \mathbb{F}_q[x]$

Internal: 27th root of unity, $W, W^2 \in \mathbb{F}_{q^{12}}$, first round challenge, $z \xleftarrow{\$} \mathbb{F}_q$

Output: $pairing(P,Q) \stackrel{?}{=} 1$

Write $s = 6t+2 = \sum_{i=0}^{L-1} s_i 2^i$ where $s_i \in \{-1,0,1\}$ and $s_L = 1$

$c^{-1} \leftarrow 1/c$ ▷ Inverse Residue witness

$f \leftarrow c^{-1}$ ▷ Initialize with inverse residue witness

$T \leftarrow Q$ ▷ T_i for each pair for multi-pairing

$x \xleftarrow{\$} \mathbb{F}_q$ ▷ Sample evaluation point from field

$e_{\text{acc}} \leftarrow 0$, $a_{\text{rlc}} \leftarrow 1$ ▷ Evaluation accumulator and random linear combination multiplier

Every operation involving T, P or Q is repeated for T_i, P_i and Q_i for multi-pairing.

Miller loop bits

for $i = L-2$ to 0 **do**

  $L_{\text{dbl}} \leftarrow l_{T,T}(P)$, $T \leftarrow [2]T$

  $R \leftarrow \mathcal{R}.pop\_front()$ ▷ Fetch R from prover

  if $s_i = 0$ **then**

    $e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * EvalPoly(R, x, f, f, L_{\text{dbl}})$ ▷ Algo 5, add L_{*i} for each P_i, Q_i

  else if $s_i = 1$ **then**

    $L_{\text{add}} \leftarrow l_{T,Q}(P)$, $T \leftarrow T + Q$

    $e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * EvalPoly(R, x, f, f, c^{-1}, L_{\text{dbl}}, L_{\text{add}})$ ▷ Algo 5, add L_{*i} for each P_i, Q_i

  else if $s_i = -1$ **then**

    $L_{\text{add}} \leftarrow l_{T,-Q}(P)$, $T \leftarrow T - Q$

    $e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * EvalPoly(R, x, f, f, c, L_{\text{dbl}}, L_{\text{add}})$ ▷ Algo 5, add L_{*i} for each P_i, Q_i

  end if

  $f \leftarrow R$, $a_{\text{rlc}} \leftarrow a_{\text{rlc}} \cdot z$ ▷ Update random linear combination factor

end for

Miller loop Frobenius correction

$R \leftarrow \mathcal{R}.pop\_front()$ ▷ Fetch R from prover

$Q_1 \leftarrow \pi_p(Q), Q_2 \leftarrow \pi_p^2(Q)$

$L_\pi \leftarrow l_{T,Q_1}(P)$

$T \leftarrow T + Q_1$

$L_{-\pi^2} \leftarrow l_{T,-Q_2}(P)$

$e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * EvalPoly(R, x, f, L_\pi, L_{-\pi^2})$ ▷ Algo 5, add L_{*i} for each P_i, Q_i

$f \leftarrow R$, $a_{\text{rlc}} \leftarrow a_{\text{rlc}} \cdot z$ ▷ Update random linear combination factor

Final exponentiation: Cubic scaling + Frobenius mappings

$R \leftarrow \mathcal{R}.pop\_front()$ ▷ Fetch R from prover

if $w = 0$ **then**

  $e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * EvalPoly(R, x, f, c^{-q}, c^{q^2}, c^{-q^3})$ ▷ Algo 5

else if $w = 1$ **then**

  $e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * EvalPoly(R, x, f, W, c^{-q}, c^{q^2}, c^{-q^3})$ ▷ Algo 5

else if $w = 2$ **then**

  $e_{\text{acc}} \leftarrow e_{\text{acc}} + a_{\text{rlc}} * EvalPoly(R, x, f, W^2, c^{-q}, c^{q^2}, c^{-q^3})$ ▷ Algo 5

end if

return $Q_{\text{acc}}(x) \stackrel{?}{=} e_{\text{acc}} \wedge R \stackrel{?}{=} 1$ ▷ Polynomial identity and expected pairing output check

n is the number of pairings. For BN254 with $k = 66$ and $n = 3$, this yields the 77% reduction observed in Table 7.

5.7 Security Analysis

We analyze soundness against a malicious prover who submits an incorrect remainder $R'_j = R_j + \Delta_j$, $\Delta_j \neq 0$.

5.7.1 Algebraic Impossibility of Forgery

Consider a malicious prover attempting to cheat by submitting an incorrect remainder $R'_j = R_j + \Delta_j$, where $\Delta_j \neq 0$ and $\deg(\Delta_j) \leq \deg(R_j) \leq 11$. Since z is obtained after the commitment to $\mathcal{R}$, the prover cannot adjust other R_k to compensate for Δ_j. To successfully deceive the verifier, the prover must find a valid forged quotient $Q'_{\text{acc}} \in \mathbb{F}_q[x]$ such that:

$$\sum_{i=0}^{n-1} z^i (A_i - R'_i) = Q'_{\text{acc}} \cdot P_{12}$$

Substituting $R'_j = R_j + \Delta_j$ and rearranging the terms yields the required relation for the forged quotient:

$$Q'_{\text{acc}} \cdot P_{12} = Q_{\text{acc}} \cdot P_{12} + z^j \Delta_j$$
$$Q'_{\text{acc}} = Q_{\text{acc}} + \frac{z^j \Delta_j}{P_{12}}$$

Because P_{12} is an irreducible polynomial with $\deg(P_{12}) = 12$, and $\deg(\Delta_j) \leq 11 < 12$, the fraction $z^j \Delta_j / P_{12}$ yields a rational function, not a polynomial. Consequently, no valid polynomial $Q'_{\text{acc}} \in \mathbb{F}_q[x]$ can exist that satisfies the verifier's equation. Algebraic forgery is thus **information-theoretically impossible**.

5.7.2 Soundness

As seen above, algebraic forgery is impossible. Therefore, security reduces to analysing the probability that a malicious prover succeeds through random chance.

Our random linear combination batching scheme (as described in Section 5.1) effectively converts our univariate polynomial verifications into a single bivariate polynomial identity test:

$$P_{\text{final}}(z, x) = \sum_{i=0}^{n-1} z^i \cdot [\prod_j P_{i,j}(x) - R_i(x)] - Q_{\text{acc}}(x) \cdot P_{12}(x)$$

If a prover commits to incorrect remainder(s), P_{err} becomes a non-zero bivariate polynomial with:

- Degree in z: at most $n - 1$ (from the RLC structure)
- Degree in x: less than 2^8 (from the polynomial products)

By the Schwartz-Zippel lemma (Section 2.2), the probability that P_{err} evaluates to zero at randomly chosen $(z, x) \in \mathbb{F}_q^2$ is bounded by:

$$\delta_s = \Pr[P_{\text{err}}(z, x) = 0] \leq \frac{\deg_z(P_{\text{err}}) + \deg_x(P_{\text{err}})}{|\mathbb{F}_q|} \leq \frac{n + 2^8}{|\mathbb{F}_q|}$$

Concrete parameters for BN254: With $n = 67$ equations (66 Miller loop steps plus 1 final exponentiation equation) and $|\mathbb{F}_q| \approx 2^{254}$:

$$\delta_s \leq \frac{67 + 256}{2^{254}} < \frac{2^9}{2^{254}} = 2^{-245}$$

Non-interactive proof via Fiat-Shamir: When transformed into a non-interactive proof with Fiat-Shamir under random oracle model, the knowledge soundness error becomes [BSCS16]:

$$\delta'_s = \delta_s + 3(m^2 + 1) \cdot 2^{-\lambda}$$

where $m = 2$ (hash queries) and $\lambda = 254$ (hash output length). Thus:

$$\delta'_s \leq 2^{-245} + 15 \cdot 2^{-254} < 2^{-244}$$

5.7.3 Completeness

For an honest prover executing Algorithm 4 with valid inputs:

1. **Polynomial construction**: At each operation, the prover computes following polynomials:
$$\prod_j P_{i,j} = Q_i \cdot P_{12} + R_i$$

2. **RLC formation**: Given challenge z, the prover forms:
$$Q_{\text{acc}} = \sum_{i=0}^{n-1} z^i \cdot Q_i$$

3. **Verifier's check**: The verifier evaluates at point x:
$$\begin{aligned}
\text{LHS} &= \sum_{i=0}^{n-1} z^i \cdot \left[\prod_j P_{i,j}(x) - R_i(x)\right] \\
&= \sum_{i=0}^{n-1} z^i \cdot Q_i(x) \cdot P_{12}(x) \quad \text{(by construction)} \\
&= P_{12}(x) \cdot \sum_{i=0}^{n-1} z^i \cdot Q_i(x) \\
&= P_{12}(x) \cdot Q_{\text{acc}}(x) = \text{RHS}
\end{aligned}$$

Since the equality holds for any evaluation point x, the protocol satisfies perfect completeness.

6 Polynomial Ring Toolkit

The protocol of Section 5 is packaged as a reusable toolkit in the `emulated` package of gnark [Con], parameterised by any emulated field $\mathbb{F}_q$ (via `FieldParams` type T) and any modulus polynomial $P \in \mathbb{F}_q[x]$. The toolkit operates in a recursive proof setting where the prover's computations are performed outside the circuit via `hints` (Section 6.2). And verifier's checks are implemented as constraints inside the circuit.

6.1 Ring Representation

A ring element is essentially a coefficient slice `[]*Element[T]` in ascending degree order. A `nil` entry is treated as zero by `InnerProductNoReduce`—the term is skipped entirely—so sparse polynomials carry no wasted constraint variables.

The optional `modEvalFn` of type `EvalFnType[T]` = (`[*Element[T]]`) → `*Element[T]` allows handling small-integer coefficients efficiently by multiplying by the small positive integer first and negating afterwards, rather than multiplying by the full-width field representative $q - c$.

Example: BN254 $\mathbb{F}_{p^{12}}$

$P_{12}(x) = x^{12} - 18x^6 + 82$ has nine zero coefficients (stored as `nil`) and two small non-zero ones. The `modEvalFn` computes $P_{12}(\alpha) =$ `MulConst`$(\alpha^0, 82)+$`Neg(MulConst`$(\alpha^6, 18))+\alpha^{12}$, keeping intermediate values small instead of multiplying by $q - 18$.

6.2 Hint Functions

Hints allow defining computations outside of a circuit. A hint defines an annotated function in the circuit at compile time. Inputs and outputs are injected at the solving time. We define these two hints to implement the prover's steps in the two-round protocol of Section 5.4.

`polyRingMulHint` implements PolyMulDiv (Algorithm 3): it multiplies the input polynomials, divides by P to obtain (Q, R), and serialises the result as `nbQTerms` | Q limbs | `nbRemTerms` | R limbs. Inputs are packed as `nbBits` | `nbLimbs` | `nbPoly` | q | inputs | P, where each polynomial is prefixed by its term count.

`quotientsRLCHint` computes the accumulated quotient $Q_{\text{acc}}[j] = \sum_i z^i \cdot Q_i[j] \bmod q$ coefficient-wise, corresponding to the prover's step in Algorithm 4. Batching all quotients into a single Q_{acc} is the key saving: only one polynomial is committed instead of s, giving the 75% calldata reduction noted in Section 4.1.2.

6.3 API

Register a ring group.

```
group := f.NewPolyRingCheck(mod, modEvalFn)
```

Creates a `PolyRingGroupChecks[T]`, appends it to the field's deferred check list, and returns a handle shared by all multiplications under the same modulus. Pass `nil` for

`modEvalFn` to use the dense fallback. Multiple groups with different moduli can coexist in one circuit.

Submit a multiplication.

```
rem, err := f.MulPolyRings(group, A, B, C, ...)
```

Computes $R = (\prod_j A_j) \bmod P$ via `polyRingMulHint` outside the circuit, range-checks R's limbs inside (`packLimbs` with `strict=true`), and registers the check $(A, B, C; R; Q)$ in `group`. The quotient Q is not range-checked; it is consumed only by the RLC hint outside the circuit. The returned `rem` is a fully reduced `*Poly[T]` and can be fed directly into the next `MulPolyRings` call.

Deferred verification.

```
f.performDeferredRingChecks(api)    // called once at circuit finalisation
```

Implements Equation (5) via two `Committer.Commit` calls (producing challenges z and x) and one `AssertIsEqual` per group. No in-circuit work beyond remainder range-checks is performed until this point.

6.4 Usage in Practice

Register the ring group once at initialisation:

```
fp, _ := emulated.NewField[emulated.BN254Fp](api)
modPoly := fp.MakePoly([]any{82, 0, 0, 0, 0, 0, -18, 0, 0, 0, 0, 0, 1})
ring := fp.NewPolyRingCheck(modPoly, nil)
```

`MakePoly` accepts Go integer literals and maps zeros to `nil`, so the slice is sparse from construction.

Each Miller-loop step then calls:

```
rem, _ = f.MulPolyRings(ring, f, f, L1, L2, L3)
```

and `performDeferredRingChecks` is invoked once by the gnark backend at finalisation, running the full two-round protocol of Section 5.

6.5 Protocol and Code Flow

The following paragraphs trace each step of the two-round protocol (Section 5.4) to the concrete function that implements it.

The verifier is essentially the circuit which encodes all the operations into constraints which can be later verified. The prover is essentially the computation done outside the circuit via `hints`.

Prover computes Q_i, R_i (Algorithm 4)

Each `MulPolyRings` calls `polyRingMulHint`, results are provided on call, Q, R for the operation stored.

Round I: commit to $\{R_i\}$, receive challenge z

`performDeferredRingChecks` collects all R_i limbs across all groups and calls `Committer.Commit` to obtain z in the circuit.

Prover forms $Q_{\text{acc}} = \sum_i z^i Q_i$ (Algorithm 4)

z from the circuit is passed to `quotientsRLCHint`, accumulating quotients coefficient-wise in mod q arithmetic.

Round II: commit to Q_{acc}, receive challenge x

The Q_{acc} limbs (together with z) are collected and passed to `Committer.Commit` to obtain x.

Verifier evaluates each polynomial at x (Algorithm 5)

`evalPolyWithChallenge` calls `InnerProductNoReduce` over the pre-computed power vector $\{x^i\}$; the result is memoised in `Poly.evaluation` to avoid re-evaluation.

Verifier checks Equation (5): $\sum_i z^i(\prod_j A_{i,j}(x) - R_i(x)) = Q_{\text{acc}}(x) \cdot P(x)$. `InnerProductNoReduce` on the per-check defects weighted by $\{z^i\}$ gives `lhs`; `MulNoReduce` of $Q_{\text{acc}}(x)$ and $P(x)$ (via `modEvalFn` or the dense fallback) gives `rhs`; `AssertIsEqual(lhs, rhs)` closes the circuit.

6.6 Implementation Details and Performance

6.6.1 Schoolbook vs Horner's Method for Polynomial Evaluation

Horner's method for evaluations across emulated elements causes a linear increase in the limbs count per coefficient. This necessitates reductions after each multiplication. Instead, we cache powers of the evaluation point x in $\mathcal{X} = (x^0, x^1, \ldots, x^d)$ and evaluate each polynomial as an inner product of $\mathcal{X}$ and the coefficient array $\mathbf{F}$:

$$\mathcal{X} \cdot \mathbf{F} = \mathcal{X}_0 \cdot F_0 + \mathcal{X}_1 \cdot F_1 + \cdots + \mathcal{X}_d \cdot F_d$$

In the emulated field context, Eval in Algorithm 5 is replaced by this inner product with $\mathcal{X}$. This allows us to use the inner product optimization described below.

6.6.2 Native field operations for inner products

Modular reductions can be deferred or entirely elided for operations expressible as inner products, such as polynomial evaluations and random linear combinations. This optimization leverages the capacity headroom provided by representing small emulated limbs (e.g., 64-bit limbs) within a significantly larger native field element.

Deferred Reduction: Intermediate scalars are permitted to grow over the integers $\mathbb{Z}$ during limb-wise multiplications and linear accumulations.

Preservation of Identity: Algebraic identities are strictly preserved provided the maximum accumulated limb magnitude remains bounded below the native characteristic p.

Handling Asymmetry: In circuits with asymmetric multiplicative depths, operands may accumulate as disparate polynomial multiples of the emulated characteristic p_{emul}. Consequently, a terminal single-element reduction suffices for equivalence testing, bypassing the constraint overhead of intermediate normalization.

7 Conclusion

7.1 Target Environments

Our approach delivers maximum impact in constraint-based environments where inversions are cheap and calldata is expensive. **Arithmetized systems** (Cairo VM, R1CS, Plonkish) benefit because Schwartz-Zippel evaluation is efficient and field inversions cost only 2C. **Calldata-constrained blockchains** (Ethereum L2s, BitVM) benefit most directly: the 77% commitment reduction means more proofs fit within transaction size limits. **Recursive proof systems** benefit because the savings compound at each recursion level. **ZKVMs** (RISC Zero, SP1, Jolt) benefit naturally as they express computation as constraint systems compatible with our polynomial framework.

7.2 Implementation and Performance

Table 7 presents measurements from Garaga's production Cairo implementation on Starknet. Our theoretical analysis (Section 4.1.2) predicted 46% constraint reduction and 75% commitment reduction per zero-bit step. The measured end-to-end results for 3-pair BN254 Miller loop are: **45% MULMOD reduction** (14,456→7,975), **77% POSEIDON reduction** (3,758→876), and **53% VM cycle reduction**.

The hash reduction slightly exceeds the per-step prediction because non-zero bits save even more commitments (two line functions instead of one per pair). The commitment savings also scale with n: traditional schemes require $O(k \cdot n)$ Fiat-Shamir commitments; our scheme requires only $O(k)$, independent of the number of pairs.

7.3 Comparison with State of the Art

For reference, Gnark—currently the leading framework for field emulation in R1CS—reports 960,784 constraints for a single BN254 pairing and 311,672 for the Miller loop alone. Our Cairo implementation achieves 53,130 steps for a single Miller loop and 155,366 for a complete pairing, reflecting a fundamentally different algorithmic approach rather than just implementation tuning.

The key structural difference is that Gnark verifies each emulated $\mathbb{F}_{q^{12}}$ multiplication individually inside the circuit, while our scheme batches all polynomial verification into a single bivariate check outside the inner circuit—with only the remainder polynomials committed to. This is why our commitment overhead (POSEIDON hashes) drops by 77% while constraint counts drop by a more modest 45%: we are eliminating a different class of cost.

Table 7: Performance comparison: Cairo implementation on Starknet

Operation	**MULMOD**		**ADDMOD**		**POSEIDON**		**Speedup**
	Before	After	Before	After	Before	After	
Miller Loop (BN254)							
Single pair	5,984	3,303	5,927	3,228	1,810	828	47.7%
Three pairs	14,456	7,975	14,463	7,924	3,758	876	53.2%
Average	**44.8%**		**45.2%**		**76.7%**		**50.6%**
Complete Pairing (BN254)							
Single pair	10,670	7,984	13,150	10,446	3,741	2,759	23.8%
Three pairs	19,142	12,656	21,686	15,142	5,689	2,807	37.1%
Average	**33.9%**		**30.2%**		**50.7%**		**30.9%**
Miller Loop (BLS12_381)							
Single pair	4,936	2,672	4,966	2,686	1,580	790	47.2%
Three pairs	11,356	6,164	11,608	6,364	3,088	834	53.8%
Average	**45.7%**		**45.5%**		**73.0%**		**50.6%**
Complete Pairing (BLS12_381)							
Single pair	10,064	7,795	14,027	11,742	3,913	3,123	19.6%
Three pairs	16,484	11,287	20,669	15,420	5,421	3,167	32.7%
Average	**31.5%**		**25.4%**		**41.6%**		**26.4%**

MULMOD/ADDMOD: Field operations in $\mathbb{F}_p$
POSEIDON: Hash operations for Fiat-Shamir;
Speedup: VM Cycle reduction.

Porting our IOP to Gnark is a direction for future work. The main challenge is that Gnark circuits allow only a single hash batching point, while our two-round IOP requires two sequential Fiat-Shamir challenges. One approach is an in-circuit GKR proof for the inner verification; we leave this as an open problem.

7.4 Final Remarks

We have shown that choosing the right granularity for polynomial identity testing—entire Miller loop steps rather than individual multiplications—yields substantial and measurable improvements. The key technique, collapsing a composite multi-operand product into a single Euclidean division check, is simple to implement and broadly applicable to other multi-step cryptographic computations in arithmetized environments.

Our implementation in Garaga is deployed on Starknet and has already enabled proof verification workloads previously infeasible on-chain. We hope this work encourages further exploration of polynomial batching granularity as a design axis for constraint-system implementations.

References

[BGM+10] Jean-Luc Beuchat, Jorge Enrique González-Díaz, Shigeo Mitsunari, Eiji Okamoto, Francisco Rodríguez-Henríquez, and Tadanori Teruya. High-speed software implementation of the optimal ate pairing over barreto-naehrig

curves. In *Pairing-Based Cryptography – Pairing 2010*, volume 6487 of *Lecture Notes in Computer Science*, pages 21–39. Springer, 2010.

[BGW+22] Dan Boneh, Sergey Gorbunov, Riad S. Wahby, Hoeteck Wee, Christopher A. Wood, and Zhenfei Zhang. BLS Signatures. Internet-draft, Internet Engineering Task Force, June 2022. Work in Progress.

[BSCS16] Eli Ben-Sasson, Alessandro Chiesa, and Nicholas Spooner. Interactive oracle proofs. In *Theory of Cryptography – TCC 2016-B*, volume 9986 of *Lecture Notes in Computer Science*, pages 31–60. Springer, 2016.

[Con] Consensys. gnark emulated fields. https://github.com/Consensys/gnark/tree/master/std/algebra/emulated.

[DHM04] Scott Vanstone Darrel Hankerson and Alfred Menezes. *Guide to Elliptic Curve Cryptography*. Springer-Verlag, 2004. Non-adjacent form representation.

[DL78] Richard A. Demillo and Richard J. Lipton. A probabilistic remark on algebraic program testing. *Information Processing Letters*, 7(4):193–195, 1978.

[FE] Feltroidprime and Keep Starknet Strange Exploration. Garaga: State-of-the-art elliptic curve tooling and snarks verification for cairo and starknet. https://github.com/keep-starknet-strange/garaga.

[Fel23] Feltroidprime. Faster extension field multiplications for emulated pairing circuits. https://hackmd.io/@feltroidprime/B1eyHHXNT, 2023. HackMD.

[FS87] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *Advances in Cryptology — CRYPTO' 86*, pages 186–194, Berlin, Heidelberg, 1987. Springer Berlin Heidelberg.

[GGPR13] Rosario Gennaro, Craig Gentry, Bryan Parno, and Mariana Raykova. Quadratic span programs and succinct nizks without pcps. In *Advances in Cryptology - EUROCRYPT 2013*, pages 626–645. Springer, 2013. Foundational QAP framework for R1CS.

[GPR21] Lior Goldberg, Shahar Papini, and Michael Riabzev. Cairo – a turing-complete STARK-friendly CPU architecture. Cryptology ePrint Archive, Paper 2021/1063, 2021.

[Gro16] Jens Groth. On the size of pairing-based non-interactive arguments. In *Advances in Cryptology – EUROCRYPT 2016*, volume 9666 of *Lecture Notes in Computer Science*, pages 305–326. Springer, 2016.

[GWC19] Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. Plonk: Permutations over lagrange-bases for oecumenical noninteractive arguments of knowledge. https://eprint.iacr.org/2019/953, 2019. Cryptology ePrint Archive, Paper 2019/953.

[Hou23] Youssef El Housni. Pairings in rank-1 constraint systems. In *Topics in Cryptology – CT-RSA 2023*, volume 13871 of *Lecture Notes in Computer Science*, pages 321–345. Springer, 2023.

[KZG10] Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In Masayuki Abe, editor, *Advances in Cryptology - ASIACRYPT 2010*, pages 177–194, Berlin, Heidelberg, 2010. Springer Berlin Heidelberg.

[NE24] Andrija Novakovic and Liam Eagen. On proving pairings. https://eprint.iacr.org/2024/640, 2024. Cryptology ePrint Archive, Paper 2024/640.

[Sch79] Jacob T. Schwartz. Probabilistic algorithms for verification of polynomial identities. In Edward W. Ng, editor, *Symbolic and Algebraic Computation*, pages 200–215, Berlin, Heidelberg, 1979. Springer Berlin Heidelberg.

[Sch80] J. T. Schwartz. Fast probabilistic algorithms for verification of polynomial identities. *J. ACM*, 27(4):701–717, October 1980.

[Zip79] Richard Zippel. Probabilistic algorithms for sparse polynomials. In Edward W. Ng, editor, *Symbolic and Algebraic Computation*, pages 216–226, Berlin, Heidelberg, 1979. Springer Berlin Heidelberg.